

Three.jsの概要と 開発環境構築

ブラウザで動く3Dグラフィックスの世界へようこそ

Week 1

2 hours

AGENDA

本日のアジェンダ

01

WebGLとThree.jsの関係

公式ドキュメントの読み方も含めて概要を把握

30 min

02

開発環境の構築

Node.js / Vite / npm install three / ES Modules

30 min

03

ハンズオン演習

Viteで青いキャンバスを表示するまで

50 min

解説 60 min → ハンズオン 50 min → 振り返り 10 min

WebGL とは



定義

ブラウザ上でGPUを使って
2D/3Dグラフィックスを描画する
JavaScript API

OpenGL ES 2.0 ベース

HTML5 <canvas> 要素で動作

1

プラグイン不要

モダンブラウザ標準搭載

2

GPU アクセラレーション

高速な並列描画処理

3

クロスプラットフォーム

PC・スマホ・タブレット対応

Three.js とは

WebGLを簡単に扱うための **JavaScript ライブラリ**

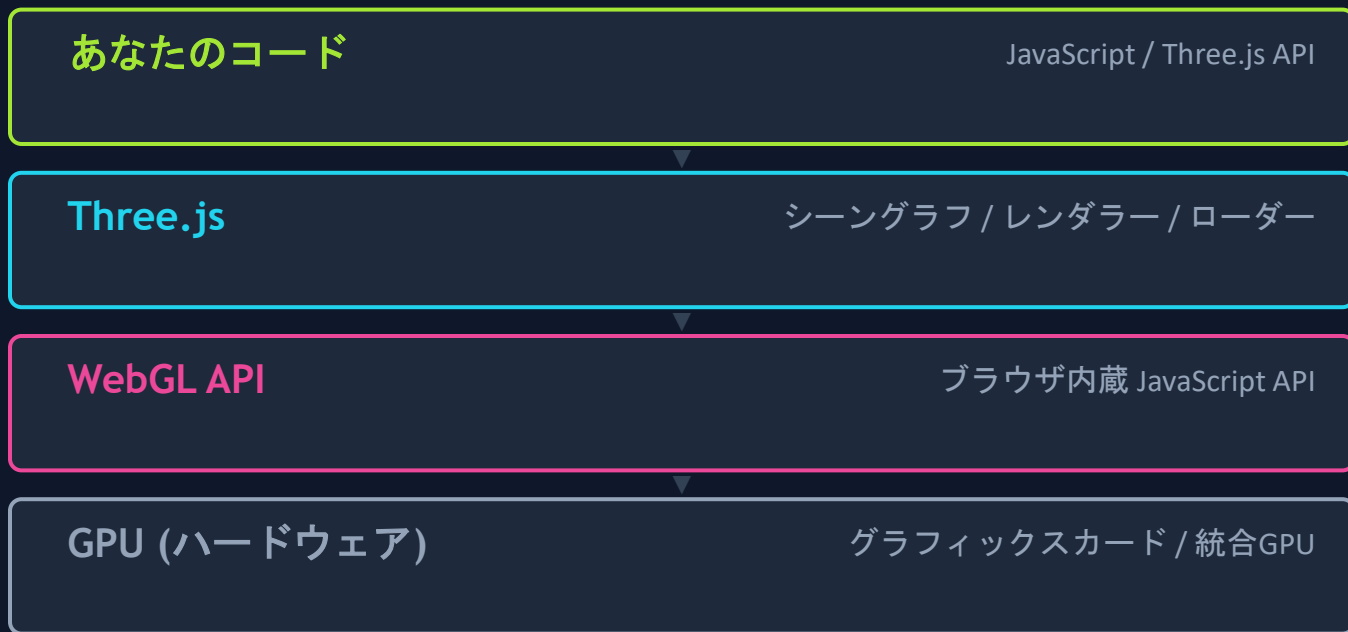
素のWebGL

- シェーダーを手書き
- 行列計算を自分で実装
- 数百行で立方体1つ

Three.js

- シーン・カメラ・メッシュの概念
- 豊富なジオメトリ・マテリアル
- 数十行で3Dシーン完成

WebGL × Three.js の関係



Three.js でできること



3D ビジュアライゼーション

データの立体的な可視化
地球儀・建築・分子構造



インタラクティブ Web

マウス・タッチで操作できる
没入型ウェブ体験



ゲーム & アニメーション

ブラウザ上で動く
リアルタイム3Dゲーム



プロダクト展示

EC サイトの3D商品ビューア
360°回転・拡大縮小

公式ドキュメントの読み方



threejs.org

Getting Started

インストール・最初のシーン作成チュートリアル。まずはここから。

API Reference

全クラスの詳細仕様。引数・戻り値・サンプルコード付き。

Examples

150以上の公式サンプル。ソースコード付きで動作確認可能。

Manual

カメラ・ライト・テクスチャなどトピック別の解説ガイド。

02

開発環境構築

Development Environment Setup

開発環境の全体像



Node.js 20+

JavaScript 実行環境



npm

パッケージ管理



Vite

開発サーバ & ビルド



VS Code

コードエディタ



Chrome

WebGL2 対応ブラウザ



Git

バージョン管理

Node.js & npm の準備



インストール確認

```
$ node --version
```

```
v20.x.x
```

```
$
```

```
npm --version
```

```
10.x.x
```

ポイント

- ✓ nodejs.org から LTS 版をダウンロード
- ✓ nvm (Node Version Manager) での管理を推奨
- ✓ npm は Node.js に同梱 — 別途インストール不要

TOOL

Vite とは

次世代フロントエンド ビルドツール

1

爆速の起動

ES Modules ベースで
バンドル不要の開発サーバ

2

ホットリロード (HMR)

ファイル保存で即座に
ブラウザに反映

3

ゼロ設定

設定ファイル不要で
すぐに使える

Webpack と比べて 10~100 倍高速な開発サーバ起動

プロジェクト作成手順

1. Viteプロジェクトを作成

```
$ npm create vite@latest my-threejs-app -- --template vanilla
```

2. ディレクトリに移動

```
$ cd my-threejs-app
```

3. 依存関係をインストール

```
$ npm install
```

4. 開発サーバを起動

```
$ npm run dev
```

Three.js のインストール

```
$ npm install three
```

何が起きるか

- `package.json` dependencies に "three" が追加される
- `node_modules/three` Three.js のソースコードがダウンロードされる
- `package-lock.json` 依存関係のバージョンが固定される

💡 CDN (importmap) でも利用可能だが、本講座では npm + Vite を推奨

ES Modules でのインポート

```
main.js
// Three.js 本体
import * as THREE from 'three';

// 個別クラスのインポート
import { Scene, PerspectiveCamera, WebGLRenderer } from 'three';

// アドオン (例: OrbitControls)
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
```

ポイント

- ✓ import / export 構文 = ES Modules (ブラウザネイティブ対応)
- ✓ Vite が自動的にモジュール解決 & バンドル
- ✓ addons/ 以下にOrbitControlsなど拡張機能が格納

最小コード構成

ファイル構成

my-threejs-app/

├─ index.html

├─ main.js

├─ style.css

├─ package.json

└─ node_modules/

└─ three/

各ファイルの役割

index.html

<canvas> を含む HTML
Vite のエントリポイント

main.js

Three.js のシーン構築
レンダリングループ

style.css

canvas の全画面表示
margin/padding リセット

ブラウザ DevTools で WebGL 確認

Console タブ

Three.js のバージョン確認
`console.log(THREE.REVISION)`

WebGL コンテキスト

`canvas.getContext('webgl2')`
が null でないことを確認

Performance タブ

FPS の計測・ボトルネック発見
GPU タスクの可視化

ショートカット: **F12** or **Cmd + Option + I** (Mac) / **Ctrl + Shift + I** (Win)

▶ HANDS-ON

Viteで青いキャンバスを表示しよう

Step 1 Vite プロジェクトを新規作成
`npm create vite@latest → vanilla` テンプレート

Step 2 Three.js をインストール
`npm install three`

Step 3 index.html を編集
`<canvas id="c">` を配置、`style.css` で全画面化

Step 4 main.js でシーンを構築
Scene → Camera → Renderer → 背景色を青に設定

Step 5 ブラウザで確認
`npm run dev → localhost:5173` で青いキャンバスが表示
ゴール: 青い背景の `<canvas>` がブラウザ全画面に表示される

まとめ & 次回予告

今日のまとめ

- ✓ WebGL は GPU を使ったブラウザ描画 API
- ✓ Three.js は WebGL の抽象化ライブラリ
- ✓ Vite + npm で開発環境を構築
- ✓ ES Modules で Three.js をインポート

次回: SESSION 02

基本構造と最初の3Dシーン

Scene / Camera / Renderer の三要素を理解し、回転する立方体を表示します